Smart Laundry: Campus Laundry Management and Tracking

Abstract

Campus laundry services require coordination between students, hostel staff, laundromats, and delivery personnel. Smart Laundry implements a web application that connects these participants through registration, collection, bag identification, delivery, and historical records. Five portals provide interfaces for administrators, wardens, laundromats, handlers, and students. React and TypeScript implement the browser application, while a Go HTTP server validates requests and manages transactions in a SQLite database. Students receive printable QR codes based on immutable roll numbers. Handlers scan these codes or enter roll numbers to collect bags and record delivery outcomes. Transactional custody records prevent duplicate collection and overlapping delivery reservations. Failed deliveries remain reserved until the handler confirms their return, after which bags become available for another attempt without losing history. Saved notifications and Server-Sent Events keep student views updated. Verification examines administrative integrity, complete laundry cycles, failure recovery, validation, and event streaming, with execution evidence distinguished from source inspection. Authentication is being integrated separately; its documented design uses password verification, server sessions, and role scoped access. This report presents project objectives, architecture, workflows, interface screenshots, an IEEE structured requirements specification, implementation details, testing methodology, and a tabular feature comparison. Remaining evaluation concerns include authentication integration, camera testing, and operational deployment.

1 Introduction

1.1 Problem context

A campus laundry service operates across several organizational boundaries. A student belongs to a hostel, a hostel is associated with a laundromat, and a handler transports bags between those locations. Each participant needs a consistent account of who owns a bag and which service event currently involves it. Registration alone cannot answer whether a bag was collected, returned, or left undelivered, but reliable registration is the foundation for those answers.

Informal approaches such as paper registers and separately maintained lists can require repeated manual reconciliation. A misspelled student name, reused identifier, or missing hostel assignment can create ambiguity when bags change hands. These are motivating scenarios for this project; no field survey or measured reduction in lost bags is claimed. Smart Laundry addresses the coordination problem through shared records, scheduled jobs, explicit bag custody, and retained history across collection, delivery, failure, and retry.

1.2 Project objectives

The first objective is to establish a common record model for laundromats, hostels, students, and handlers. The student roll number also identifies the associated bag. Keeping this identity immutable prevents an edit from silently transferring a bag reference to another student. Archiving a removed student reserves that identity and preserves links to historical laundry attempts.

The second objective is to support each participant through an appropriate portal. Administrators manage campus records and wardens register hostel students. Laundromats schedule pickups and deliveries, assign handlers, and inspect batches. Handlers start assigned jobs, identify bags, record handovers, and complete routes. Students display their bag QR code and consult current status, upcoming work, notifications, and completed history. Identity selection in the operational views is being replaced or constrained by separately integrated authentication.

The third objective is to maintain an explicit custody lifecycle. A pickup begins with a hostel and handler assignment; confirmed bag scans form the pickup batch. Only bags from a completed pickup become eligible for delivery. Delivery selection reserves bags atomically, so a bag cannot belong to two outstanding deliveries. Successful handover removes the bag from custody. A failed handover retains its reservation until route completion confirms return to the laundromat, then permits a new attempt.

The fourth objective is to make the workflow observable and durable. Job records, item results, custody, and notifications persist in SQLite. Server-Sent Events tell connected clients to reload their snapshots when data changes. A student can therefore distinguish collection, storage at the laundromat, scheduled delivery, and a failed delivery returning to the laundromat. This is an in-app update mechanism rather than a background push or SMS service.

1.3 Contributions and scope

The project contributes a operational workflow on top of reusable administrative record management. Shared Go handlers validate identities and parent assignments. Transactions protect bulk registration, delivery reservation, and bag state transitions. Historical results remain available after later attempts, allowing a failed delivery and a successful retry to be represented independently.

The interface contribution is a coordinated set of browser portals, including mobile-oriented student and handler views. Camera scanning includes a manual roll-number alternative, while the student portal generates a printable QR code. Batch filtering helps laundromats choose collected bags for return. The design contribution is a requirements baseline that connects these behaviors to acceptance checks and distinguishes authentication integration from tested laundry behavior.

The project excludes payments, washing-machine control, garment-level inventory, and route optimization. It does not establish measured reductions in lost bags or service time. These benefits require a campus pilot with representative users. Authentication is described from the emerging session-based implementation and remains subject to integration verification. This report evaluates the operational application without treating an unfinished security integration as a verified result.

2 Techniques/Technologies Used

2.1 Technology selection and responsibilities

React provides the component and state model for the five browser portals. TypeScript describes record shapes and catches compilation errors before deployment. Vite produces the production JavaScript, CSS, and font assets. Bun installs dependencies and runs the frontend build; Inter fonts and Lucide icons provide a consistent visual vocabulary. These interface choices support reusable dialogs and record lists across administrative records and operational views. Camera frames are decoded using jsQR; qrcode.react generates student bag codes.

The backend uses Go's standard HTTP server and method aware routing. JSON request handlers perform validation, look up active parent records, execute SQL, and return consistent responses. SQLite is accessed through Go's database interface and the pure Go SQLite driver. The repository also contains sqlc configuration and generated query bindings, but the administrative handlers currently execute SQL directly. Describing sqlc as the active administrative data layer would overstate its role.

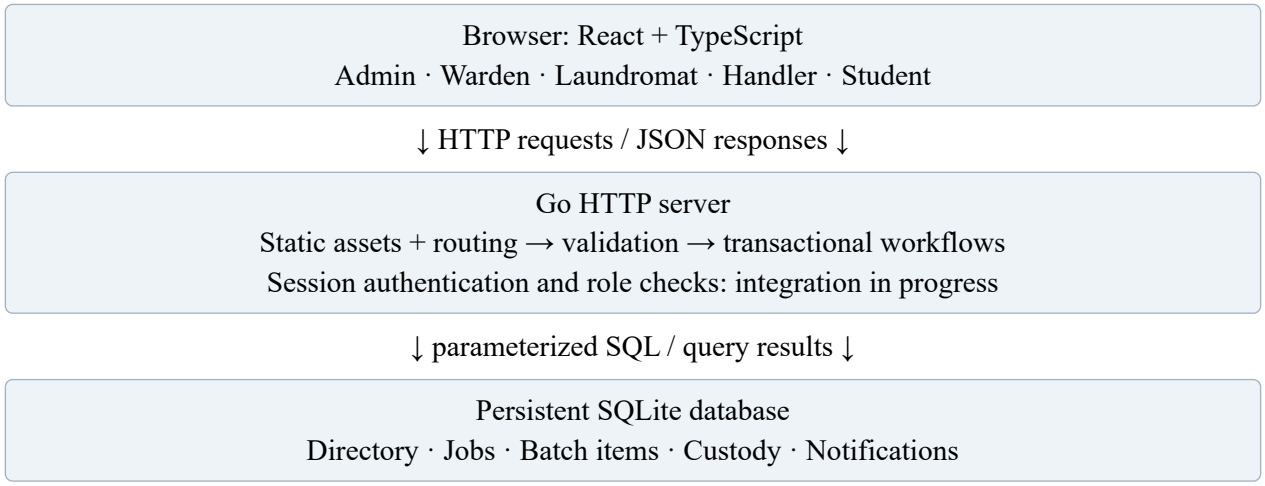


Figure 1: Implemented application architecture. All five portals and the API share the Go server; SSE triggers snapshot refresh.

2.2 Data integrity techniques

The server trims names and accepts between one and 120 Unicode characters. Student roll numbers must be positive integers within JavaScript's safe integer range. Non-null parent identifiers must resolve to active records. Parameter placeholders bind supplied values, while resource table names come from a fixed server side map. These decisions keep validation consistent across resource types.

A transaction groups each write's checks and updates. Bulk creation shares one transaction across up to 1,000 input objects, so a later conflict rolls back earlier inserts. Removal sets an archive flag and clears dependent administrative assignments. Operational transactions also bind job results, bag custody, and saved notifications. A delivery reservation rolls back completely if any selected bag is unavailable, and a repeated scan is rejected rather than creating a second handover.

2.3 Development and operational workflow

The frontend build runs TypeScript checking before Vite bundles the application. Backend tests use isolated SQLite databases and HTTP request recorders; a live HTTP test exercises event streaming. The multi-stage Dockerfile builds frontend assets and the Go executable, then copies both into a runtime image. Compose attaches a persistent SQLite volume. `DB_PATH` and `PORT` configure database location and listening port.

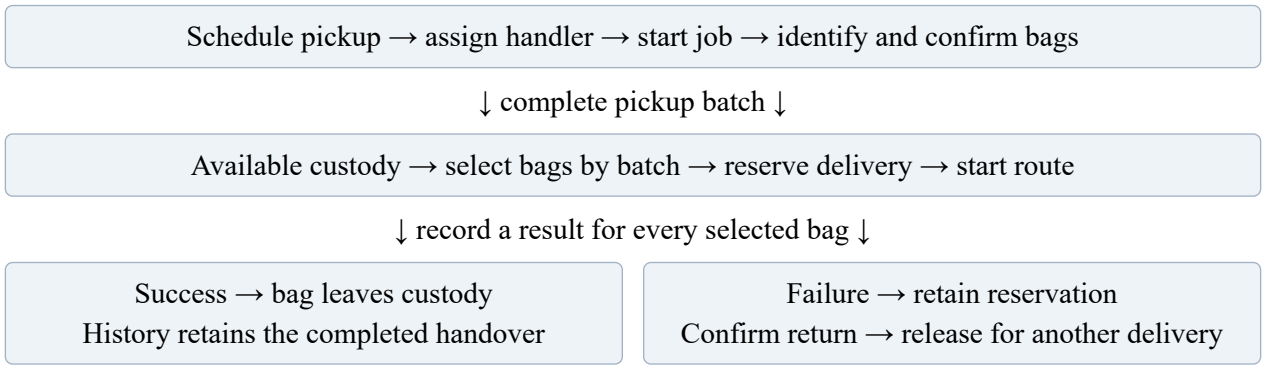


Figure 2: Implemented laundry workflow, including the failed-delivery return and retry path.

State-changing transactions commit before publishing live events. Clients reconnect to the same-origin SSE stream and reload persisted snapshots, allowing notifications to be recovered after reconnection. SQLite uses a single open connection and a busy timeout to serialize operations consistently. A persistent file replaces the earlier disposable in-memory database.

3 Design & SRS

3.1 Interface design and implemented features

The administrator view presents counts, searchable tables, individual and bulk dialogs, and archival removal. The warden view focuses on hostel registrations. The laundromat portal schedules work, assigns handlers, filters collected bags by pickup batch, and reserves delivery selections. Handler views expose assigned jobs and bag actions; student views combine a printable QR code, current custody status, saved notifications, and history. Shared visual controls make state and errors visible across portals.

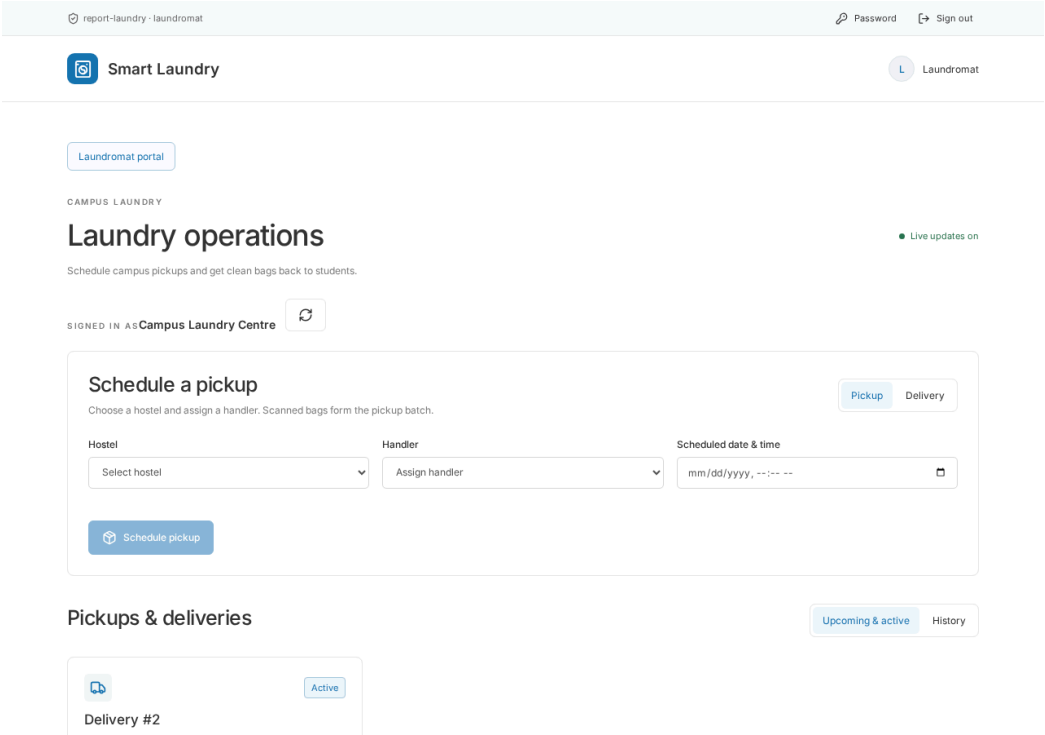


Figure 3: Laundromat portal with synthetic demonstration data: scheduling, assignments, and pickup/delivery jobs.

3.2 Software requirements specification

Document control and organization. SRS version 2.0, operational baseline, 5 November 2026. Organization follows IEEE 830-1998 [1]; identified, verifiable requirements are informed by ISO/IEC/IEEE 29148:2018 [2]. Sources are the stakeholder outline and inspected code [3]. This is a project SRS rather than a certification of conformance.

1. Introduction. 1.1 Purpose and scope: Specify campus registration, bag pickup, custody, delivery, notifications, and history for developers and evaluators. Payments and machine control are excluded. **1.2 Definitions:** SRS: Software Requirements Specification; QR: Quick Response; batch: bags collected in one pickup; custody: an undelivered bag held by the service; SSE: Server-Sent Events. **1.3 References:** [1]–[3] follow the conclusion.

2. Overall description. 2.1 Perspective and functions: A shared browser/server application combines directory management with transactional laundry jobs. **2.2 User classes:** Admin manages records; warden registers hostel students; laundromat schedules and reserves; handler scans and records outcomes; student views their bag and history. **2.3 Environment and constraints:** Modern browser, network, Go server, persistent SQLite, and same-origin JSON/SSE. Camera capture requires HTTPS or localhost and permission; manual input provides a fallback. One roll number identifies one bag. Authentication integration is a dependency of enforced account isolation. **2.4 Assumptions:** Campus operators maintain correct assignments, and handlers confirm physical handovers and returned failed bags.

3. Specific requirements. All listed functions are mandatory (M). I denotes implemented behavior; A denotes authentication integration still requiring verification. Acceptance methods are criteria, not a claim that every boundary has been tested.

3.1 External interfaces. UI-01: The system shall provide the five role portals and show action errors without discarding the context. IF-01: Writes shall accept JSON; validation, missing-record, and conflict responses shall use 400, 404, and 409. IF-02: The scanner shall recognize `smart-laundry:<roll_number>` and numeric roll input. IF-03: Connected clients shall refresh on SSE change events and reload saved notifications on reconnection.

ID	Functional requirement and acceptance method	State
FR-01	The system shall create, list, update, and archive campus records. Check CRUD responses, retained IDs, and detached children.	M/I
FR-02	The system shall preserve roll-number identity and reject reuse after archival. Check duplicate creation and immutable updates.	M/I
FR-03	The system shall create 1–1,000 bulk records atomically. Introduce a late conflict and compare database state.	M/I
FR-04	The warden view shall register and remove students for its hostel. Check single and bulk assignments; server scope depends on auth.	M/I
FR-05	The system shall schedule a hostel pickup, assign a handler, and collect each eligible bag once. Reject repeated and wrong-hostel scans.	M/I
FR-06	The system shall reserve selected bags only after pickup completion. Attempt overlapping and partially invalid reservations.	M/I
FR-07	The system shall record delivery outcomes and reject completion with pending bags. Verify successful handover removes custody.	M/I
FR-08	Failed bags shall remain reserved until return is confirmed at route completion, then become available for retry. Verify failure and retry history.	M/I
FR-09	The system shall retain student history and saved notifications, including failed attempts. Verify filtered snapshots and mark-read behavior.	M/I
FR-10	The system shall authenticate accounts, derive their role and subject from a session, and reject unauthorized access. Run role, expiry, logout, and CSRF tests after integration.	M/A

3.2 Quality and data requirements. NFR-01: Committed records shall survive process restart; compare file-backed snapshots before and after reopening. NFR-02: Names shall contain 1–120 characters; non-null parent IDs shall identify active records. NFR-03: Unassigned parent selections shall submit null. NFR-04: Credentials shall use salted password hashes and sessions shall expire and be revocable (A). NFR-05: A 100-student list shall render within two seconds in 95 of 100 trials in a documented environment; performance evaluation remains pending. Job items shall retain distinct delivery attempts and timestamps. Data retention duration requires campus approval before real deployment.

4. Verification and traceability. Registration needs map to FR-01–04 and TC-01; transport needs map to FR-05–08 and TC-02–04; visibility maps to FR-09 and TC-05–06. FR-10 maps to the separate authentication acceptance checks. Security integration, physical camera evaluation, and performance measurement remain explicit acceptance activities. Changes require a revised baseline and matching checks.

4 Implementation

The browser application selects a portal from its URL. Shared request helpers serialize JSON and display server errors. Administrative state maintains entity lists, searches, filters, and dialogs. Operational state combines directory data with jobs, custody, and notifications. The laundromat view schedules jobs; the handler view confirms identified bags; and the student view derives status from current custody and job results. Operational identity preferences are remembered locally while account integration is being completed.

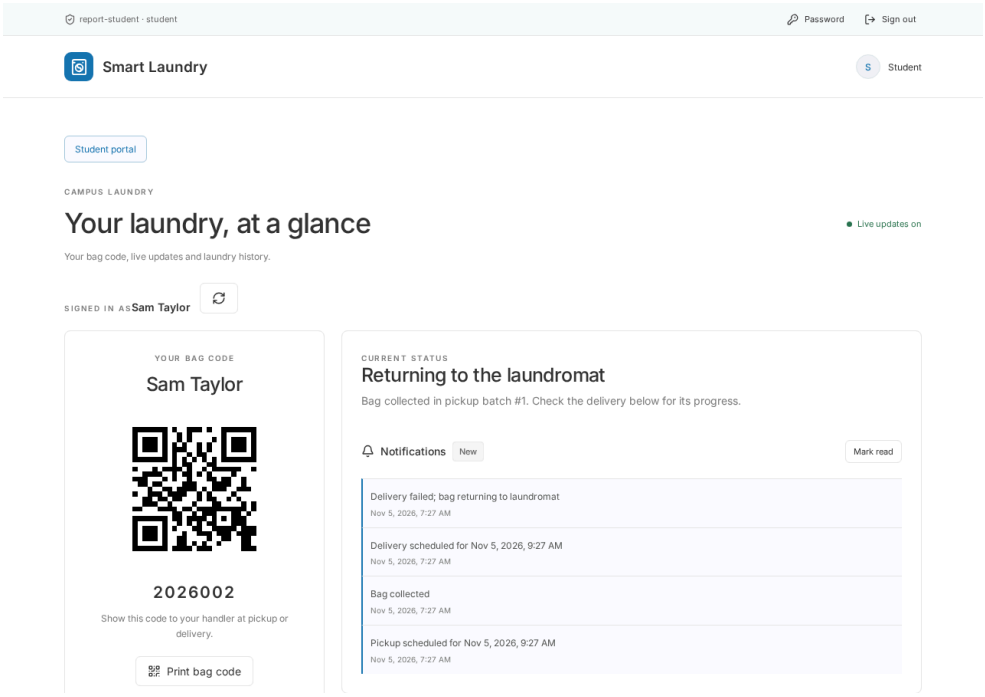


Figure 4: Student portal showing the generated bag QR code, current state, saved notifications, and laundry work.

The Go API uses shared administrative handlers and separate workflow handlers. Persistent tables `laundry_job`, `laundry_item`, `laundry_custody`, and `notification` hold operational data. The original pickup/delivery tables remain for compatibility. Transactions prevent partial bulk creation, repeated collection, and overlapping delivery reservation. A failed delivery keeps custody reserved until route completion confirms return; its item result remains in history when a new job succeeds. Notifications are saved within the transaction, and a published SSE event causes connected portals to refresh.

Camera capture draws frames to a canvas for jsQR decoding. A recognized code populates the roll-number field; the handler confirms collection or delivery separately. Manual input follows the same API path. Student codes encode `smart-laundry:<roll_number>` and can be printed. This identifies a bag without embedding credentials in its QR payload.

Authentication design—separate integration. The source being integrated uses username/password login, salted PBKDF2 password verification, and random server sessions stored by token hash in SQLite. An HttpOnly cookie carries the session; role and linked student, handler, hostel, or laundromat determine permitted access. Middleware checks sessions and requires a CSRF token for writes. Logout revokes the session. These details describe the inspected integration design; successful login and role isolation are not claimed without passing the separate auth checks. `DB_PATH`, `PORT`, and a Compose volume configure durable deployment.

5 Testing

5.1 Methodology and execution evidence

Verification combines Go integration tests, TypeScript checking and Vite bundling, and browser/API checks against an isolated demonstration database. The backend fixtures initialize the real schema and route requests through the application handlers. Workflow cases exercise complete pickup/delivery paths, state validation, failed delivery recovery, history, and notification read state. A live HTTP test checks the SSE connection, change delivery, and subscriber cleanup. The generated database helper package has no direct tests.

All four backend tests passed in the recorded run, and the frontend production build passed. Authentication is being integrated concurrently; the live checks additionally verified role-based access to the account list. Test assertions about a supplied handler ID establish workflow assignment checks; authenticated identity enforcement additionally requires session and role tests. Compilation establishes that frontend types and assets build, while camera accuracy and device usability require separate evaluation.

Case	Scenario and expected outcome	Evidence / requirement
TC-01	CRUD, duplicate roll, archive identity, bulk conflict and rollback	Admin integration; FR-01–04
TC-02	Start pickup; reject wrong handler, wrong hostel, duplicate scan and invalid state	Workflow cycle; FR-05
TC-03	Reject delivery before pickup completion and overlapping reservation; roll back partial selection	Workflow cycle; FR-06
TC-04	Reject pending completion; record failure; block early retry; return then retry successfully	Workflow cycle; FR-07–08
TC-05	Retain failed attempt, filter student history, persist and mark notifications read	Workflow cycle; FR-09
TC-06	Open SSE stream, publish change, disconnect and remove subscriber	Live-stream test; IF-03
TC-07	Persist state to disk and reopen server with the same database	Live reopen check; NFR-01
TC-08	Authenticated roles, session expiry, logout, and CSRF rejection	Separate auth acceptance; FR-10

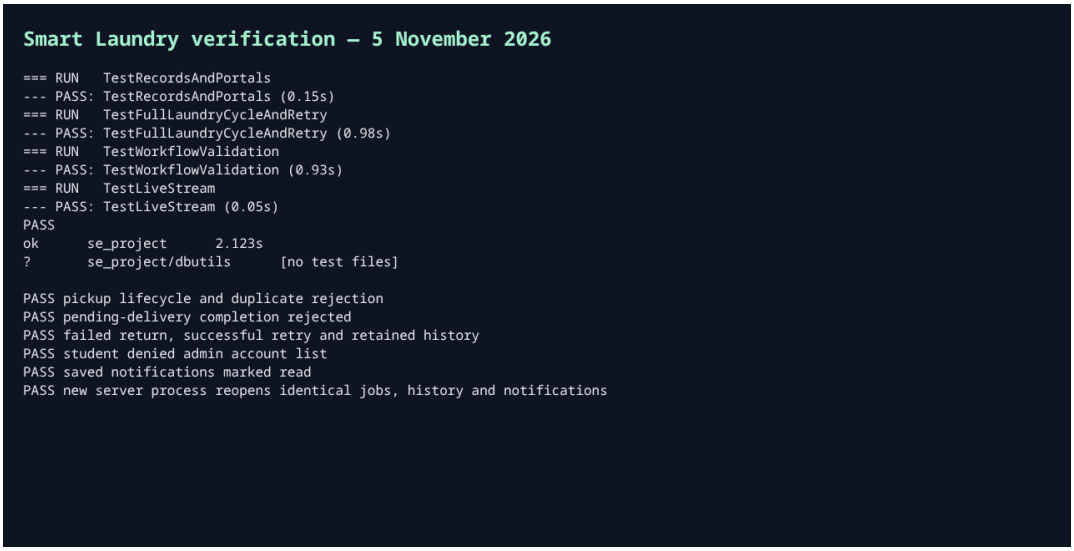


Figure 5: Recorded test output and live workflow checks displayed in a browser evidence view. Authentication integration status is recorded separately.

5.2 Interpretation

Checks distinguish transaction integrity from physical operations. The application can reject a duplicate bag confirmation and preserve a failed attempt, but a handler must still confirm the actual handover or return. The retry path explicitly tests the interval between failed handover and return confirmation. No code-coverage percentage, campus performance result, or measured reduction in lost bags is inferred from passing functional checks.

5.3 Interface evidence and feature comparison

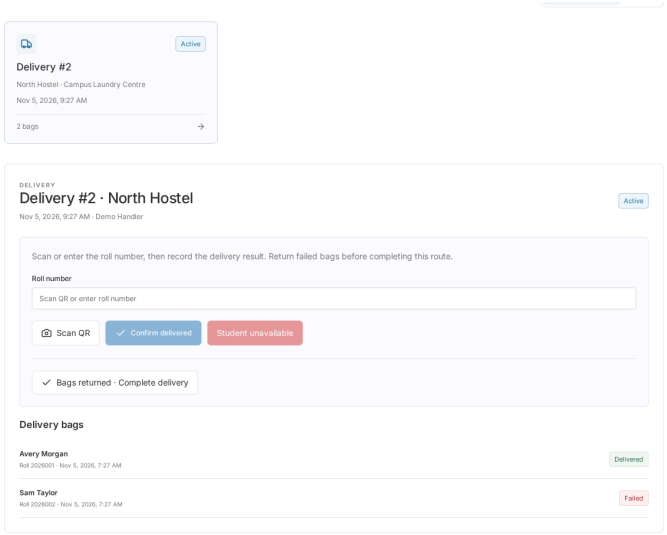


Figure 6: Handler portal with job details, bag confirmation controls, and recorded outcomes. Camera and manual identification share the confirmation workflow.

The handler interface exposes the scheduled → active → completed lifecycle and displays each bag's result. Delivery completion requires all assigned bags to be processed. After a failure, the completion label asks the handler to confirm that bags were returned. Students can inspect failed and successful attempts in history. Screenshots use synthetic records; no real student data is included.

The comparison uses a paper register and a basic spreadsheet without custom scripts as analytical feature baselines. These are existing record-management approaches rather than measured commercial competitors. Spreadsheet capabilities vary with additional rules and integrations.

Feature	Paper register	Basic spreadsheet	Smart Laundry
Registration	Manual entries	Rows and filters	Shared records; hostel filters
Identity checks	Manual lookup	Configured validation	Immutable rolls; duplicate rejection
Bulk integrity	Manual reconciliation	Depends on extra tooling	Atomic validation and rollback
Bag identification	Visual/manual lookup	Separate scanning tool	Generated QR, camera and manual entry
Collection/delivery	Handwritten updates	Manually maintained status	Jobs, batches and custody transitions
Failed return	Manual follow-up	Custom columns/process	Reserved until return; retained retry history
Notifications	Calls or messages	Requires integration	Saved in-app messages; SSE refresh
Persistence	Retained pages	Saved file	SQLite file and Compose volume
Account access	Physical process	File sharing permissions	Session/role integration in progress

5.4 Remaining evaluation

Authentication acceptance must verify each role against unauthorized routes and records, including student history and notifications, cross-hostel registration, and handlers acting on another person's jobs. It must also check missing/expired sessions, logout revocation, password changes, and missing CSRF tokens. These outcomes belong to the separate auth integration and are not inferred from workflow assignment checks.

Camera evaluation should cover real printed codes, lighting, permission refusal, and manual fallback on representative phones. A pilot should additionally measure concurrent registration, recovery from interrupted requests, backup restoration, SSE reconnection, and usability. Durable storage and workflow checks support the completed application; final deployment acceptance also requires those operational and security evaluations.

Conclusion

Smart Laundry now implements the campus laundry lifecycle through five coordinated browser portals. Administrators maintain campus records, wardens manage hostel registration, laundromats schedule jobs, handlers collect and deliver bags, and students access bag codes, status, notifications, and history. Immutable roll numbers and archival records preserve identity, while persistent SQLite storage retains operational information across application restarts.

The implementation makes custody explicit. Confirmed pickup scans form batches, completed pickups release bags for delivery selection, and reservations prevent overlapping assignments. A successful delivery removes a bag from custody. A failed attempt remains recorded and reserved until return confirmation makes the bag available for another delivery. Saved notifications and live events connect these state changes to student views.

Evaluation combines administrative and workflow integration tests, event stream checks, frontend compilation, and recorded browser/API scenarios. Each result is interpreted within its tested scope; camera performance, realistic workload behavior, and campus usability require additional evaluation. Authentication is being integrated separately using password verification, session cookies, and role based authorization, with its acceptance dependent on dedicated security checks.

The operational system provides a basis for a campus pilot. Subsequent work should validate account isolation, deployment recovery, and device usability, then measure service outcomes using student workloads.

References

[1] IEEE, *IEEE 830-1998: Recommended Practice for Software Requirements Specifications*. IEEE standard record. Used for SRS organization.

[2] ISO/IEC/IEEE, *29148:2018: Systems and software engineering—Life cycle processes—Requirements engineering*. ISO standard record. Used for requirement identification and verification principles.

[3] Project repository, inspected 5 November 2026: `docs/SRS.md` (stakeholder needs); `backend/admin.go`, `main.go`, `frontend.go`, `sql/schema.sql`, and `admin_test.go`, `workflow.go`, `workflow_test.go`, and the evolving `auth.go`; `frontend/src/main.tsx`, `portals.tsx`, `package.json`; `backend/Dockerfile` and `compose.yaml`. Execution evidence: `report/evidence/backend-tests.txt`, `frontend-build.txt`, and `browser-checks.json`. Screenshots use synthetic demonstration records.